Image Classifier Project Tutorial (12th Class Friendly)

1) Big Picture: What is this project?

Think of this project like teaching a student to identify animals in photos.

Here, the student is a neural network model.

Input: a photo

Output: class name (example: cats or dogs) + confidence score

This project is not just training. It gives a full production pipeline:

- training code
- saved best model
- command-line prediction
- visual dashboard
- web API for real applications

So this is a complete mini-product, not only an experiment.

2) Project structure and purpose of each part

- README.md: quick overview and how to start
- PRODUCTION_REPORT.md: deployment-oriented report
- PROJECT_TUTORIAL_12TH_GUIDE.md: this detailed student guide
- requirements.txt: Python packages needed
- dashboard.py: Streamlit dashboard for visual interaction
- src/data.py: loading data + image transforms
- src/model.py: model factory (ResNet50/EfficientNet/custom CNN)
- src/train.py: full training loop
- src/inference.py: command-line predictions
- [src/api.py](src/api.py): FastAPI server for production-style prediction endpoint
- reports/training_log.csv: per-epoch metrics generated by training
- reports/run_config.json: exact training settings used
- checkpoints/best_model.pt: best trained model
- sample_data: example dataset in class-folder format

3) Data format: how images must be arranged

This project uses ImageFolder style. It means class names are folder names.

Example:

sample_data/

cats/

cat1.jpg

cat2.jpg

dogs/

dog1.jpg

dog2.jpg

Rule:

- one folder per class
- images inside each class folder

Why this is useful:

- no manual label file required
- PyTorch automatically maps class name to number

4) End-to-end flow

1. Data loaded and preprocessed by src/data.py
2. Model created by src/model.py
3. Model trained by src/train.py
4. Best weights saved as checkpoints/best_model.pt
5. Same checkpoint used by:

- src/inference.py
- dashboard.py
- [src/api.py](src/api.py)

Important idea:

- One trained checkpoint powers all prediction interfaces.
- This keeps behavior consistent between demo and production.

5) Core ML idea explained simply

5.1 Transfer learning

Training a deep model from zero needs huge data and time.

So we reuse a model already trained on ImageNet (millions of images).

You can imagine this as:

- model already knows edges, shapes, textures
- we only teach it our final task classes

This is faster and usually better for small/medium datasets.

5.2 Why normalize images?

Models expect pixel values in a similar numeric range to what they saw during pretraining.
So we normalize with ImageNet mean/std in src/data.py.

5.3 Why augmentation?

In training, images are randomly changed a little:

- random crop
- horizontal flip
- slight rotation
- color jitter

Purpose:

- reduce memorization
- improve generalization to real-world variation

6) Deep dive into each code component

6.1 src/data.py

Main responsibilities:

- build transforms for train and validation
- load dataset with torchvision.datasets.ImageFolder
- split into train/val with fixed seed
- return DataLoaders and class mapping

Functions:

- build_transforms(image_size, train)

- train=True: random augmentations + normalization
- train=False: deterministic resize + normalization

- get_dataloaders(data_dir, image_size, batch_size, val_split, seed)

- creates dataset

- performs reproducible split
- ensures validation has no random augmentation
- returns:
- train_loader
- val_loader
- class_to_idx

Why DataLoader?

- gives batches (example: 32 images at once)
- faster and memory-efficient

6.2 src/model.py

Main function: build_model(arch, num_classes, freeze_backbone=True)

Supports:

- resnet50
- efficientnet_b0
- custom_cnn

For pretrained models:

- if freeze_backbone=True, earlier layers are frozen
- final classifier layer is replaced to match your class count

Simple interpretation:

- old knowledge kept
- final decision head retrained for your classes

6.3 src/train.py

This is the training engine.

Key stages:

1. Parse command arguments (data_dir, arch, epochs, etc.)
2. Build dataloaders
3. Build model
4. Define loss (CrossEntropyLoss)
5. Define optimizer (Adam)
6. Define scheduler (ReduceLROnPlateau)
7. Loop over epochs:

- train batches
- evaluate on validation
- log metrics
- save best checkpoint
- stop early if no improvement for patience window

Saved outputs:

- checkpoints/best_model.pt
- reports/training_log.csv
- reports/run_config.json

Metrics:

- val_accuracy
- val_f1 (macro)

Why macro F1?

- better when class counts are imbalanced
- gives equal importance to each class

6.4 src/inference.py

Purpose:

- run predictions from terminal
- one image or a whole folder

Flow:

1. load checkpoint metadata + model weights
2. rebuild same model architecture
3. apply same validation transform
4. run forward pass
5. convert logits to probabilities with softmax
6. print predicted class and confidence

Important folder note:

- folder input should contain image files directly
- example: sample_data/cats
- not the class-parent folder when script expects flat files

6.5 dashboard.py

Purpose:

- non-technical visual interface

Sections in dashboard:

1. Training Performance

- best validation accuracy
- final loss
- epoch count
- line charts from training log

2. Try the Model

- upload image
- see predicted class and confidence
- see all class probabilities as chart

Technical highlights:

- uses cache to avoid reloading model repeatedly
- reads checkpoint and training log from project-root paths

6.6 [src/api.py](src/api.py)

Purpose:

- expose model through HTTP so apps can call it

Endpoints:

- GET /health

- service status and loaded classes

- POST /predict

- accepts image file
- returns JSON with:
- predicted_class
- confidence
- all_probabilities

Why API matters:

- website/mobile/backend systems can use model predictions programmatically

7) Math idea behind classification (basic)

The model outputs a score vector called logits:

z = [z1, z2, ..., zk]

Softmax converts scores to probabilities:

p_i = exp(z_i) / sum(exp(z_j))

Predicted class is index with highest probability:

predicted_class = argmax(p_i)

Cross-entropy loss compares predicted probabilities with true label.

Lower loss means better predictions.

8) How to run everything (base Python, no venv activation)

Run from project root.

1. Install requirements once:

C:/Python314/python.exe -m pip install -r requirements.txt

2. Train:

C:/Python314/python.exe src/train.py --data_dir sample_data --arch resnet50 --epochs 15

3. Dashboard:

C:/Python314/python.exe -m streamlit run dashboard.py --server.port 8501

4. CLI inference (folder example):

C:/Python314/python.exe src/inference.py --checkpoint checkpoints/best_model.pt --folder sample_data/cats

5. API:

C:/Python314/python.exe -m uvicorn src.api:app --host 0.0.0.0 --port 8000

6. API quick health test:

Open http://127.0.0.1:8000/health

9) Understanding training logs

In reports/training_log.csv:

- epoch: training cycle number

- train_loss: loss on training data
- val_loss: loss on validation data
- val_accuracy: percent of correct validation predictions
- val_f1: macro F1 score
- lr: learning rate used in that epoch
- seconds: epoch duration

Healthy signs:

- val_accuracy rises and stabilizes
- val_loss generally decreases

Warning signs:

- train improves but val stagnates or drops (overfitting)
- very unstable metrics (possible bad data or settings)

10) Common problems and fixes

Problem: ModuleNotFoundError for data/model

- Cause: import path mismatch
- Fix in this project: src is package-enabled and imports are standardized

Problem: dashboard not opening

- Cause: wrong port or stale old streamlit process
- Fix: run with fixed port and open exact URL

Problem: API works but inference fails on folder

- Cause: folder contains subfolders, not image files directly
- Fix: point to a class folder or flat image folder

Problem: low accuracy

- Try:
- more data
- better class balance
- tune epochs/lr
- switch architecture
- unfreeze backbone when enough data is available

11) How to explain this project in interviews

Short version:

- Built a complete image classification system using transfer learning
- Added reproducible training pipeline with metrics logging and early stopping
- Delivered three inference interfaces: CLI, dashboard, and FastAPI
- Ensured one shared checkpoint powers all interfaces for consistent behavior

12) Next learning upgrades

If you want to improve this project academically and professionally:

- add confusion matrix and per-class precision/recall report
- add test dataset separated from validation
- add model versioning and experiment tracking
- add automated tests for API and inference
- containerize with Docker and add CI pipeline

13) Final takeaway

This project teaches a full AI product lifecycle:

- data preparation
- model design and training
- evaluation
- user-facing demo
- production API deployment

If you understand each file in this guide, you understand not just one script, but how ML systems are built for real use.